

Algorithms Mid-term Exam

Take-home (40%) Due: 2026/05/08

Contents

1	Question 1: Modified Merge-Sort with Insertion-Sort Subroutine	2
2	Question 2: Recursive Binary Search	5
3	Question 3: Substitution Method for $T(n) = T(n - 1) + \Theta(n)$	7
4	Question 4: The Anatomy of Sorting Algorithm Execution	9

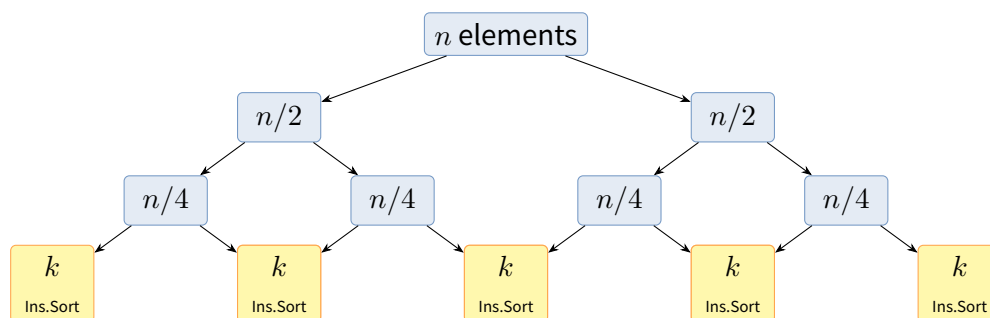
1 Question 1: Modified Merge-Sort with Insertion-Sort Subroutine

Question

Consider a modification to MERGE-SORT in which n/k sublists of length k are sorted using INSERTION-SORT and then merged using the standard merging mechanism, where k is a value to be determined.

- (a) Show that the n/k sublists, each of length k , can be sorted by INSERTION-SORT in $\Theta(nk)$ worst-case time.
- (b) Show that the sublists can be merged in $\Theta(n \lg(n/k))$ worst-case time.
- (c) Given that the modified algorithm runs in $\Theta(nk + n \lg(n/k))$ worst-case time, what is the largest asymptotic value of k as a function of n for which the modified algorithm has the same asymptotic running time as standard MERGE-SORT?
- (d) How should k be chosen in practice?

Background. 標準的 Merge-Sort 會遞迴地將 n 個元素的陣列不斷切半，直到每個子陣列長度為 1，再逐層向上合併。改良版則把遞迴基底案例改為 Insertion-Sort：當子陣列長度縮減到 k 時停止遞迴，對每個子陣列用 Insertion-Sort 排序，再向上合併。這正是 TimSort 的核心思想。



■ Insertion-Sort on k elements — Merge-Sort divide/merge levels ($\lg(n/k)$ levels)

(a) Insertion-Sort Phase: $\Theta(nk)$ Worst-Case

Claim

Sorting n/k sublists of length k with Insertion-Sort takes $\Theta(nk)$ time in the worst case.

Proof. 回顧課堂內容：Insertion-Sort 對長度為 m 的陣列最多執行 $1+2+\dots+(m-1) = \frac{m(m-1)}{2}$ 次比較，故其最差情況複雜度為 $\Theta(m^2)$ 。

以 $m = k$ 代入每個子陣列：

- 共有 $\frac{n}{k}$ 個子陣列，每個長度為 k 。
- 每個子陣列在最差情況下耗時 $\Theta(k^2)$ 。

因此，所有子陣列的最差情況總時間為：

$$\frac{n}{k} \cdot \Theta(k^2) = \Theta\left(\frac{n}{k} \cdot k^2\right) = \Theta(nk). \quad \square$$

(b) Merge Phase: $\Theta(n \lg(n/k))$ Worst-Case

Claim

Merging the n/k sorted sublists (each of length k) takes $\Theta(n \lg(n/k))$ time.

Proof. 完成 Insertion-Sort 後，共有 n/k 個已排序子陣列。標準 Merge-Sort 的合併過程在 n/k 個子陣列上共進行

$$\lg\left(\frac{n}{k}\right)$$

(每層將子陣列數減半，直到只剩一個)。在每一層中，共有 n 個元素被逐一處理。故總合併成本為：

$$\underbrace{\Theta(n)}_{\text{per level}} \times \underbrace{\lg\left(\frac{n}{k}\right)}_{\text{levels}} = \Theta\left(n \lg \frac{n}{k}\right). \quad \square$$

(c) Largest Asymptotic k for Same Running Time as Merge-Sort

Claim

The largest asymptotic value of k (as a function of n) for which the modified algorithm achieves the same $\Theta(n \lg n)$ as standard Merge-Sort is $k = \Theta(\lg n)$.

Analysis. 改良版演算法的總執行時間為

$$T_{\text{mod}}(n) = \Theta(nk + n \lg(n/k)).$$

標準 Merge-Sort 執行時間為 $\Theta(n \lg n)$ 。我們要求 $T_{\text{mod}}(n) = \Theta(n \lg n)$ ，需滿足：

- (i) **第一項**。 $nk = O(n \lg n) \implies k = O(\lg n)$.
- (ii) **第二項**。 $n \lg(n/k) = n(\lg n - \lg k)$. 當 $k = O(\lg n)$ 時， $\lg k = O(\lg \lg n) = o(\lg n)$ ，故 $n \lg(n/k) = \Theta(n \lg n)$. ✓
- (iii) **下界**。我們需要 $nk = \Omega(1)$ (顯然成立) 以及整體表達式為 $\Theta(n \lg n)$ 。取 $k = c \lg n$ ($c > 0$ 為常數)，則 $nk = cn \lg n = \Theta(n \lg n)$ 。

因此 k 的最大漸近值為：

$$k = \Theta(\lg n).$$

(d) Practical Choice of k : The TimSort Connection

TimSort Insight

In practice, k is chosen as a small constant (32-64) that makes Insertion-Sort empirically faster than Merge-Sort on real hardware. This is precisely the strategy used by **TimSort** (Python `sort()`, Java `Arrays.sort()`).

以下三個互補理由說明為何選擇固定的小 k ，而非理論最優的 $c \lg n$ ：

1. **快取局部性 (Cache Locality)**。Insertion-Sort 依序存取連續的 k 個元素；所有資料都能放入 L1 快取，使 $\Theta(k^2)$ 中的隱藏常數 C 極小。

2. **無需額外記憶體**。Insertion-Sort 是原地排序 (in-place)：僅需 $O(1)$ 的額外空間，避免了 Merge-Sort 所需的 $\Theta(n)$ 輔助陣列配置開銷。
3. **自適應優勢**。Insertion-Sort 在部分已排序的輸入上性能優雅降級（趨近 $\Theta(k)$ 而非 $\Theta(k^2)$ ），與 Merge-Sort 善用已排序片段的特性相輔相成。

The causal chain is:

small k $\xrightarrow{\text{causes}}$ Ins-Sort stays in cache $\xrightarrow{\text{causes}}$ small constant factor C $\xrightarrow{\text{causes}}$ faster in practice.

2 Question 2: Recursive Binary Search

Question

Recursive algorithm:

- (a) Write pseudocode for Binary Search.
- (b) Write a recurrence for the time complexity $T(n)$ and represent the time complexity using asymptotic notation.

(a) Pseudocode

Algorithm

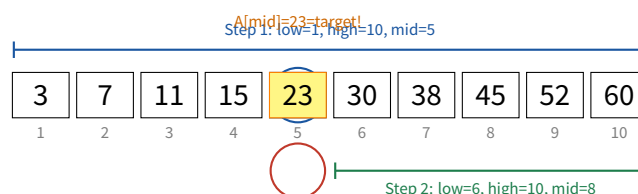
Input: Sorted array $A[1..n]$; indices low , $high$; search key $target$

Output: Index i such that $A[i] = target$, or -1 if not found

```
if  $low > high$  then
    | return  $-1$   $\triangleright$  Base case: empty sub-array
end
 $mid \leftarrow \lfloor (low + high) / 2 \rfloor$ 
if  $A[mid] = target$  then
    | return  $mid$   $\triangleright$  Found
end
else if  $A[mid] < target$  then
    | return Binary-Search( $A$ ,  $mid + 1$ ,  $high$ ,  $target$ )  $\triangleright$  Search right half
end
else
    | return Binary-Search( $A$ ,  $low$ ,  $mid - 1$ ,  $target$ )  $\triangleright$  Search left half
end
```

Algorithm 1: BINARY-SEARCH(A , low , $high$, $target$)

下圖示意在已排序陣列中搜尋 $target = 23$ 的過程。



(b) Recurrence and Asymptotic Complexity

每次呼叫中，我們將 $target$ 與 $A[mid]$ 進行一次 $\Theta(1)$ 的比較，再對大小為一半的子陣列遞迴。由此得到遞迴式：

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1), \quad T(1) = \Theta(1).$$

Solution via Master Theorem. The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$, $f(n) = \Theta(1)$.

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = \Theta(1) = f(n).$$

This matches Case 2: $f(n) = \Theta(n^{\log_b a})$. Therefore:

$$\boxed{T(n) = \Theta(\lg n)}.$$

Intuition. 每次遞迴呼叫將剩餘元素減半（因果步驟），因此經過 $\lg n$ 次對半後，搜尋空間縮減為單一元素。這正是 $\Theta(\lg n)$ 直接由對半結構得出的原因。

3 Question 3: Substitution Method for $T(n) = T(n - 1) + \Theta(n)$

Question

Use the substitution method to prove that the recurrence $T(n) = T(n - 1) + \Theta(n)$ has the solution $T(n) = \Theta(n^2)$.

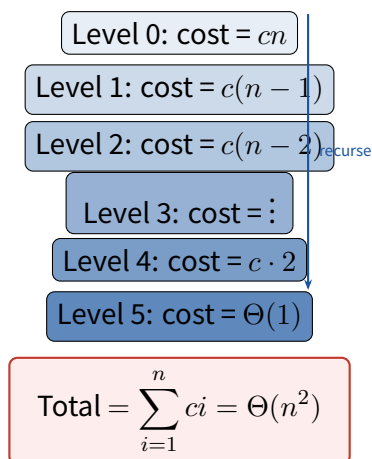
Goal

Prove that $T(n) = T(n-1) + \Theta(n)$ has solution $T(n) = \Theta(n^2)$ using the substitution method.

Motivation / Unrolling (why $\Theta(n^2)$?). 展開遞迴式可以得到直觀的圖像：

$$\begin{aligned} T(n) &= T(n-1) + cn \\ &= T(n-2) + c(n-1) + cn \\ &\vdots \\ &= T(1) + c \cdot 2 + c \cdot 3 + \cdots + cn \\ &= \Theta(1) + c \sum_{i=2}^n i = \Theta(1) + c \cdot \frac{n(n+1)}{2} - c \\ &= \Theta(n^2). \end{aligned}$$

每層「付出」的代價為 c 乘以該層的層號；總代價為 $1 + 2 + \cdots + n$ 的加總，等於 $n(n+1)/2 = \Theta(n^2)$ 。下方的代入法將此猜測嚴格形式化。



Proof: Upper Bound $T(n) = O(n^2)$ **Hypothesis:** $T(n-1) \leq c_1(n-1)^2$ for some $c_1 > 0$.**Inductive step:**

$$\begin{aligned}
T(n) &= T(n-1) + dn \\
&\leq c_1(n-1)^2 + dn \\
&= c_1n^2 - 2c_1n + c_1 + dn \\
&= c_1n^2 - (2c_1 - d)n + c_1.
\end{aligned}$$

The last expression is $\leq c_1n^2$ provided $(2c_1 - d)n \geq c_1$, i.e. $n \geq \frac{c_1}{2c_1 - d}$, which holds for all sufficiently large n whenever $c_1 > d/2$ (choose, e.g., $c_1 = d$). Hence $T(n) = O(n^2)$. $\square$

Proof: Lower Bound $T(n) = \Omega(n^2)$ **Hypothesis:** $T(n-1) \geq c_2(n-1)^2$ for some $c_2 > 0$.**Inductive step:**

$$\begin{aligned}
T(n) &= T(n-1) + dn \\
&\geq c_2(n-1)^2 + dn \\
&= c_2n^2 - 2c_2n + c_2 + dn \\
&= c_2n^2 + (d - 2c_2)n + c_2.
\end{aligned}$$

The last expression is $\geq c_2n^2$ provided $(d - 2c_2)n + c_2 \geq 0$. Choose $c_2 = d/2$; then $d - 2c_2 = 0$ and $c_2 > 0$, so $c_2n^2 + c_2 \geq c_2n^2$. $\checkmark$ Hence $T(n) = \Omega(n^2)$. $\square$

Since $T(n) = O(n^2)$ and $T(n) = \Omega(n^2)$:

$$T(n) = \Theta(n^2).$$

Lecture Connection

This recurrence arises in **Insertion Sort** and **Bubble Sort**: at each level i , the algorithm does $\Theta(i)$ comparisons/swaps. Summing over all n levels causes the $\Theta(n^2)$ total. The causal chain is:

reduce problem by 1 each step $\Rightarrow$ linear work per step $\Rightarrow$ quadratic total.

4 Question 4: The Anatomy of Sorting Algorithm Execution

Question

The Anatomy of sorting algorithms execution:

- Tight Code & Constants:** Quicksort is often praised for having "tight code" in its inner loop. Explain what "tight code" means in this context and how it directly influences the hidden constant factor (C) in the runtime equation $T(n) = C(n \log n)$. (Hint: Compare the fundamental operations inside a Quicksort partition vs. a Mergesort merge.)
- Algorithmic Strategy & Trade-offs:** You are designing a system for a Safety-Critical Medical Device where memory is extremely limited and you must guarantee that sorting never exceeds a certain time limit, regardless of how "messy" the input data is. Between Quicksort (with randomized pivot) and Heapsort, which would you choose? Justify your answer by discussing their Worst-case Time Complexity and Space Complexity.

(a) Tight Code and the Hidden Constant Factor C

Definition: Tight Code

當演算法的內層迴圈在每次迭代中執行**最少數量的基本操作**——無不必要的記憶體存取、無函式呼叫開銷、無輔助資料搬移——則稱其具備 **tight code**，使 $T(n) = C \cdot n \log n$ 中的隱藏常數 C 盡可能小。

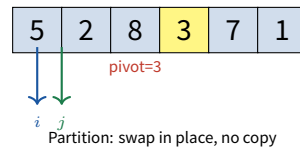
Causal mechanism. 所有基於比較的排序演算法共享下界 $\Omega(n \log n)$ (decision-tree argument)。然而，同樣是 $\Theta(n \log n)$ 的兩個演算法可能相差一個常數因子。每次比較所需執行的基本機器操作數是 C 的直接原因：

fewer ops per iteration $\xrightarrow{\text{causes}}$ smaller C $\xrightarrow{\text{causes}}$ faster wall-clock time.

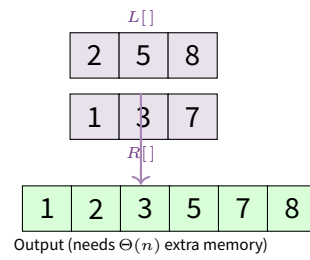
QuickSort Partition vs. Merge-Sort Merge (per element):

QuickSort Partition (tight)	Merge-Sort Merge (looser)
1 comparison: $A[j] \leq \text{pivot}?$	2 comparisons: $L[i] \leq R[j] + \text{boundary}$
1 pointer increment: $j += 1$	2 writes to auxiliary arrays $L[], R[]$
Occasional in-place swap (2 ops)	1 write-back to $A[]$ per element
No extra memory , all ops on A	$\Theta(n)$ auxiliary memory allocation
Sequential access $\Rightarrow$ cache-friendly	Two separate arrays $\Rightarrow$ cache misses
$C_{QS} \approx 1\text{--}2$	$C_{MS} \approx 3\text{--}5$

QuickSort Partition



Merge-Sort Merge



總結：Quicksort 緊湊的內層迴圈直接導致更小的常數因子 C ，這解釋了為何儘管漸近複雜度相同，Quicksort 在實際上總是優於 Merge-Sort。

(b) Safety-Critical Medical Device: QuickSort vs. Heap-Sort

System Constraints

- Extremely limited memory:** auxiliary space must be minimised.
- Hard real-time guarantee:** sorting must never exceed a fixed time bound, regardless of input.

Complexity comparison:

Algorithm	Time Complexity		Space (extra)	Deterministic?
	Average	Worst-case		
QuickSort (rand. pivot)	$\Theta(n \lg n)$	$\Theta(n^2)$	$O(n)$ stack	No
Heap-Sort	$\Theta(n \lg n)$	$(n \lg n)$	$O(1)$	Yes

Recommendation: Heap-Sort

Choose Heap-Sort. The causal justification proceeds in two steps.

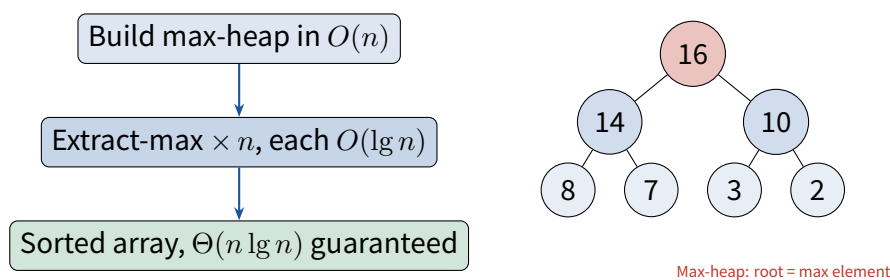
Step 1 --- Worst-case time guarantee. QuickSort's randomised pivot makes $\Theta(n^2)$ behaviour improbable but not impossible. On a safety-critical device, improbable failures are unacceptable: even a single $\Theta(n^2)$ run could miss a critical deadline.

Heap-Sort builds a **max-heap** from the array in $O(n)$ time (heapify), then performs n extract-max operations each taking $O(\lg n)$ time. The total is:

$$O(n) + n \cdot O(\lg n) = \Theta(n \lg n),$$

deterministically---the same for every possible input permutation. This is the direct causal reason the time bound can be guaranteed.

Step 2 --- Space guarantee. QuickSort's recursion stack requires $O(\lg n)$ on average but $O(n)$ in the worst case. On memory-limited embedded hardware this stack overflow risk is unacceptable. Heap-Sort operates in-place using $O(1)$ auxiliary space (the heap lives inside the original array), so memory usage is completely bounded.



Conclusion. The causal chain is:

Heap-Sort $\rightarrow$ $\Theta(n \lg n)$ worst-case and $O(1)$ space $\rightarrow$ safe, bounded execution
choice time guarantee memory guarantee system requirement met.

Quicksort，儘管擁有緊湊的程式碼與優秀的平均情況性能，但無法同時提供上述兩項保證，因此不適用於此安全關鍵情境。